

Contents

International Plebeian Academy - Comprehensive Pseudo Code	1
Revolutionary Holographic Distributed System Architecture	1
Table of Contents	1
System Initialization	1
Core Authentication & Security	2
Blockchain File Verification	3
Holographic Distribution System	5
Biometric Administration	7
Seven Division Bot System	10
Upgrade Propagation System	13
Frontend Application	17
Backend API System	24
Tribal Coin Blockchain	29
Infrastructure & Deployment	37
Summary	46

International Plebeian Academy - Comprehensive Pseudo Code

Revolutionary Holographic Distributed System Architecture

Table of Contents

1. System Initialization
 2. Core Authentication & Security
 3. Blockchain File Verification
 4. Holographic Distribution System
 5. Biometric Administration
 6. Seven Division Bot System
 7. Upgrade Propagation System
 8. Frontend Application
 9. Backend API System
 10. Tribal Coin Blockchain
 11. Infrastructure & Deployment
-

System Initialization

```
FUNCTION initialize_plebeian_academy_system():
    // Initialize core system components
    config = load_configuration_from_environment()

    // Initialize security layer
    quantum_security = initialize_quantum_key_distribution()
    biometric_system = initialize_multi_modal_biometric_system()

    // Initialize blockchain components
    file_verification_blockchain = initialize_file_verification_system()
    tribal_coin_blockchain = initialize_tribal_coin_system()

    // Initialize distribution layer
    holographic_distributor = initialize_holographic_distribution()
    torrent_network = initialize_torrent_mesh_network()
```

```

// Initialize bot management
seven_division_bots = initialize_bot_system()

// Initialize upgrade manager
upgrade_manager = initialize_upgrade_propagation_system()

// Start monitoring and health checks
start_system_monitoring()

RETURN system_instance

FUNCTION load_configuration_from_environment():
    config = {
        database_url: get_env("DATABASE_URL"),
        blockchain_provider: get_env("WEB3_PROVIDER_URL"),
        encryption_keys: get_env("ENCRYPTION_KEY"),
        biometric_enabled: get_env("BIOMETRIC_ENABLED"),
        quantum_keys_enabled: get_env("QUANTUM_KEYS_ENABLED"),
        holographic_fragments: get_env("HOLOGRAPHIC_FRAGMENTS", 7),
        minimum_seeds: get_env("MINIMUM_SEEDS", 3),
        replication_factor: get_env("REPLICATION_FACTOR", 5)
    }

    validate_configuration(config)
    RETURN config

```

Core Authentication & Security

```

FUNCTION authenticate_user(user_credentials, biometric_data):
    // Multi-layer authentication process

    // Step 1: Basic credential validation
    IF NOT validate_basic_credentials(user_credentials):
        RETURN authentication_failed("Invalid credentials")

    // Step 2: Biometric verification (if enabled)
    IF biometric_enabled:
        biometric_result = biometric_system.authenticate(user_credentials.user_id, biometric_data)
        IF NOT biometric_result.success:
            RETURN authentication_failed("Biometric verification failed")

    // Step 3: Generate quantum-secured session
    quantum_key = quantum_security.generate_session_key()
    session_token = create_secure_session_token(user_credentials.user_id, quantum_key)

    // Step 4: Log authentication event
    audit_log.record_authentication(user_credentials.user_id, timestamp(), "SUCCESS")

    RETURN authentication_success(session_token, quantum_key, biometric_result.confidence_score)

FUNCTION validate_basic_credentials(credentials):
    user = database.get_user(credentials.user_id)

```

```

IF user IS NULL:
    RETURN FALSE

password_hash = hash_password(credentials.password, user.salt)
RETURN password_hash EQUALS user.password_hash

FUNCTION create_secure_session_token(user_id, quantum_key):
    payload = {
        user_id: user_id,
        issued_at: timestamp(),
        expires_at: timestamp() + SESSION_DURATION,
        quantum_key_hash: hash(quantum_key)
    }

    token = encrypt_with_quantum_key(payload, quantum_key)
    RETURN token

```

Blockchain File Verification

```

FUNCTION register_file_on_blockchain(file_path, file_content, admin_signature):
    // Create File Command Allocation Table (FCAT) entry

    // Step 1: Calculate file hash and metadata
    file_hash = calculate_sha256(file_content)
    file_size = get_file_size(file_content)
    file_metadata = {
        path: file_path,
        hash: file_hash,
        size: file_size,
        timestamp: timestamp(),
        admin_signature: admin_signature,
        version: generate_version_number()
    }

    // Step 2: Create blockchain transaction
    transaction_data = {
        operation: "FILE_REGISTER",
        file_metadata: file_metadata,
        previous_hash: get_latest_file_hash(file_path),
        merkle_root: calculate_merkle_root([file_hash])
    }

    // Step 3: Submit to blockchain
    transaction_hash = blockchain.submit_transaction(transaction_data)

    // Step 4: Wait for confirmation
    WHILE NOT blockchain.is_transaction_confirmed(transaction_hash):
        wait(BLOCK_CONFIRMATION_TIME)

    // Step 5: Update local FCAT
    fcat_entry = {
        file_path: file_path,
        blockchain_hash: transaction_hash,

```

```

        file_hash: file_hash,
        registered_at: timestamp(),
        status: "VERIFIED"
    }

    local_fcat.add_entry(fcat_entry)

    RETURN transaction_hash

FUNCTION verify_file_authenticity(file_path, file_content):
    // Verify file against blockchain FCAT

    // Step 1: Calculate current file hash
    current_hash = calculate_sha256(file_content)

    // Step 2: Get blockchain record
    fcat_entry = local_fcat.get_entry(file_path)
    IF fcat_entry IS NULL:
        RETURN verification_failed("File not registered in FCAT")

    // Step 3: Verify against blockchain
    blockchain_record = blockchain.get_transaction(fcat_entry.blockchain_hash)
    IF blockchain_record IS NULL:
        RETURN verification_failed("Blockchain record not found")

    // Step 4: Compare hashes
    IF current_hash EQUALS blockchain_record.file_metadata.hash:
        RETURN verification_success("File verified authentic")
    ELSE:
        RETURN verification_failed("File hash mismatch - potential tampering detected")

FUNCTION create_upgrade_token(upgrade_id, file_list, admin_signature, biometric_hash):
    // Create blockchain token for worldwide upgrade authorization

    token_data = {
        upgrade_id: upgrade_id,
        files: file_list,
        admin_signature: admin_signature,
        biometric_hash: biometric_hash,
        created_at: timestamp(),
        expires_at: timestamp() + UPGRADE_TOKEN_VALIDITY
    }

    // Create smart contract transaction
    contract_call = {
        function: "createUpgradeToken",
        parameters: token_data
    }

    transaction_hash = blockchain.call_smart_contract(UPGRADE_CONTRACT_ADDRESS, contract_call)

    RETURN {
        token_id: upgrade_id,
        transaction_hash: transaction_hash,

```

```
        expires_at: token_data.expires_at
    }
```

Holographic Distribution System

```
FUNCTION initialize_holographic_distribution():
    // Initialize quantum-inspired holographic data distribution

    holographic_system = {
        fragment_count: HOLOGRAPHIC_FRAGMENTS,
        redundancy_level: REPLICATION_FACTOR,
        encoding_algorithm: "QUANTUM_INSPIRED_FRACTAL",
        mesh_nodes: [],
        active_torrents: {}
    }

    // Initialize mesh network
    mesh_network = initialize_mesh_network()

    // Start holographic encoding service
    start_holographic_encoder()

    RETURN holographic_system

FUNCTION distribute_file_holographically(file_path, file_content):
    // Distribute file using holographic encoding across mesh network

    // Step 1: Create holographic fragments
    fragments = create_holographic_fragments(file_content, HOLOGRAPHIC_FRAGMENTS)

    // Step 2: Add redundancy encoding
    redundant_fragments = add_fractal_redundancy(fragments, REPLICATION_FACTOR)

    // Step 3: Create torrent for each fragment
    torrent_hashes = []
    FOR each fragment IN redundant_fragments:
        torrent_hash = create_torrent(fragment)
        torrent_hashes.append(torrent_hash)

    // Seed fragment across mesh network
    seed_fragment_to_mesh(fragment, torrent_hash)

    // Step 4: Create holographic map
    holographic_map = {
        file_path: file_path,
        original_hash: calculate_sha256(file_content),
        fragment_hashes: torrent_hashes,
        reconstruction_algorithm: "QUANTUM_FRACTAL_DECODE",
        created_at: timestamp()
    }

    // Step 5: Distribute map across network
    distribute_holographic_map(holographic_map)
```

```

RETURN holographic_map

FUNCTION create_holographic_fragments(data, fragment_count):
    // Quantum-inspired holographic encoding

    fragments = []
    data_size = length(data)
    base_fragment_size = data_size / fragment_count

    FOR i = 0 TO fragment_count - 1:
        // Create overlapping fragments with quantum interference patterns
        start_offset = i * base_fragment_size * 0.7 // 30% overlap
        end_offset = start_offset + base_fragment_size * 1.3

        fragment_data = extract_data_slice(data, start_offset, end_offset)

        // Apply quantum-inspired encoding
        encoded_fragment = apply_quantum_encoding(fragment_data, i)

        // Add error correction
        error_corrected_fragment = add_reed_solomon_encoding(encoded_fragment)

        fragments.append(error_corrected_fragment)

    RETURN fragments

FUNCTION reconstruct_file_from_hologram(holographic_map):
    // Reconstruct original file from holographic fragments

    // Step 1: Collect fragments from mesh network
    collected_fragments = []
    FOR each torrent_hash IN holographic_map.fragment_hashes:
        fragment = download_from_torrent_mesh(torrent_hash)
        IF fragment IS NOT NULL:
            collected_fragments.append(fragment)

    // Step 2: Check if we have enough fragments
    minimum_fragments = ceil(holographic_map.fragment_hashes.length * 0.6) // 60% minimum
    IF length(collected_fragments) < minimum_fragments:
        RETURN reconstruction_failed("Insufficient fragments available")

    // Step 3: Decode fragments
    decoded_fragments = []
    FOR each fragment IN collected_fragments:
        decoded_fragment = apply_quantum_decoding(fragment)
        error_corrected = apply_reed_solomon_decoding(decoded_fragment)
        decoded_fragments.append(error_corrected)

    // Step 4: Reconstruct using quantum interference
    reconstructed_data = quantum_interference_reconstruction(decoded_fragments)

    // Step 5: Verify reconstruction
    reconstructed_hash = calculate_sha256(reconstructed_data)

```

```

IF reconstructed_hash EQUALS holographic_map.original_hash:
    RETURN reconstruction_success(reconstructed_data)
ELSE:
    RETURN reconstruction_failed("Hash verification failed")

FUNCTION seed_fragment_to_mesh(fragment, torrent_hash):
    // Seed fragment across mesh network nodes

    // Step 1: Select optimal seeding nodes
    seeding_nodes = select_optimal_nodes(MINIMUM_SEEDS)

    // Step 2: Upload fragment to selected nodes
    FOR each node IN seeding_nodes:
        upload_result = node.upload_fragment(fragment, torrent_hash)
        IF upload_result.success:
            mesh_network.record_seeder(node.id, torrent_hash)

    // Step 3: Start torrent swarm
    start_torrent_swarm(torrent_hash, seeding_nodes)

    // Step 4: Monitor seeding health
    schedule_seeding_health_check(torrent_hash)

```

Biometric Administration

```

FUNCTION initialize_multi_modal_biometric_system():
    // Initialize comprehensive biometric authentication system

    biometric_system = {
        fingerprint_processor: initialize_fingerprint_processor(),
        iris_processor: initialize_iris_processor(),
        voice_processor: initialize_voice_processor(),
        face_processor: initialize_face_processor(),
        behavioral_processor: initialize_behavioral_processor(),
        enrolled_templates: {},
        authentication_threshold: 0.7,
        modality_weights: {
            fingerprint: 0.25,
            iris: 0.25,
            voice: 0.20,
            face: 0.15,
            behavioral: 0.15
        }
    }

    RETURN biometric_system

FUNCTION enroll_admin_biometrics(admin_id, biometric_samples):
    // Enroll administrator with multiple biometric modalities

    templates = {}

    // Process each biometric modality

```

```

FOR each modality, sample_data IN biometric_samples:
    SWITCH modality:
        CASE "fingerprint":
            template = process_fingerprint_enrollment(sample_data)
        CASE "iris":
            template = process_iris_enrollment(sample_data)
        CASE "voice":
            template = process_voice_enrollment(sample_data)
        CASE "face":
            template = process_face_enrollment(sample_data)
        CASE "behavioral":
            template = process_behavioral_enrollment(sample_data)

    IF template.quality_score >= MINIMUM_QUALITY_THRESHOLD:
        templates[modality] = template

// Require minimum number of modalities
IF length(templates) >= MINIMUM_MODALITIES:
    biometric_system.enrolled_templates[admin_id] = templates
    RETURN enrollment_success()
ELSE:
    RETURN enrollment_failed("Insufficient biometric modalities")

FUNCTION authenticate_admin_biometric(admin_id, biometric_samples):
    // Authenticate administrator using multiple biometric modalities

    IF admin_id NOT IN biometric_system.enrolled_templates:
        RETURN authentication_failed("Admin not enrolled")

    enrolled_templates = biometric_system.enrolled_templates[admin_id]
    modality_scores = {}
    total_weighted_score = 0.0

    // Process each provided biometric sample
    FOR each modality, sample_data IN biometric_samples:
        IF modality IN enrolled_templates:
            // Extract features from sample
            sample_features = extract_biometric_features(modality, sample_data)

            // Compare with enrolled template
            similarity_score = compare_biometric_templates(
                sample_features,
                enrolled_templates[modality]
            )

            modality_scores[modality] = similarity_score
            weight = biometric_system.modality_weights[modality]
            total_weighted_score += similarity_score * weight

    // Determine authentication result
    IF total_weighted_score >= biometric_system.authentication_threshold:
        // Generate quantum-secured session
        quantum_key = quantum_security.generate_admin_key()
        session_token = create_admin_session_token(admin_id, quantum_key)

```



```

        RETURN authentication_success(session_token, quantum_key, total_weighted_score)
ELSE:
    RETURN authentication_failed("Biometric verification failed")

FUNCTION process_fingerprint_enrollment(fingerprint_image):
    // Process fingerprint for enrollment

    // Step 1: Enhance image quality
    enhanced_image = enhance_fingerprint_image(fingerprint_image)

    // Step 2: Extract minutiae points
    minutiae_points = extract_minutiae_points(enhanced_image)

    // Step 3: Create feature vector
    feature_vector = create_minutiae_feature_vector(minutiae_points)

    // Step 4: Calculate quality score
    quality_score = calculate_fingerprint_quality(enhanced_image, minutiae_points)

    // Step 5: Create template
    template = {
        modality: "fingerprint",
        features: feature_vector,
        quality_score: quality_score,
        created_at: timestamp(),
        template_hash: calculate_sha256(feature_vector)
    }

    RETURN template

FUNCTION extract_minutiae_points(fingerprint_image):
    // Extract ridge endings and bifurcations

    // Step 1: Apply ridge enhancement
    enhanced_ridges = apply_gabor_filters(fingerprint_image)

    // Step 2: Binarize image
    binary_image = binarize_image(enhanced_ridges)

    // Step 3: Thin ridges
    thinned_ridges = apply_ridge_thinning(binary_image)

    // Step 4: Detect minutiae
    minutiae_points = []
    FOR each pixel IN thinned_ridges:
        IF is_ridge_ending(pixel, thinned_ridges):
            minutiae_points.append(create_minutiae_point(pixel, "ending"))
        ELSE IF is_ridge_bifurcation(pixel, thinned_ridges):
            minutiae_points.append(create_minutiae_point(pixel, "bifurcation"))

    RETURN minutiae_points

```

Seven Division Bot System

```
FUNCTION initialize_bot_system():
    // Initialize comprehensive AI bot management system

    bot_system = {
        divisions: initialize_seven_divisions(),
        active_bots: {},
        task_queue: initialize_priority_queue(),
        performance_metrics: {},
        human_oversight_queue: initialize_oversight_queue()
    }

    // Start bot orchestration
    start_bot_orchestrator()

    RETURN bot_system

FUNCTION initialize_seven_divisions():
    // Create seven specialized divisions with 5 bots each

    divisions = {
        "community_engagement": {
            bots: [
                create_bot("community_moderator", "Moderate discussions and enforce guidelines"),
                create_bot("event_coordinator", "Organize and manage community events"),
                create_bot("member_onboarding", "Welcome and guide new members"),
                create_bot("feedback_collector", "Gather and analyze community feedback"),
                create_bot("social_media_manager", "Manage social media presence")
            ]
        },
        "content_management": {
            bots: [
                create_bot("content_curator", "Curate and organize educational content"),
                create_bot("quality_assessor", "Assess content quality and relevance"),
                create_bot("translation_manager", "Manage multilingual content"),
                create_bot("archive_manager", "Organize and maintain content archives"),
                create_bot("plagiarism_detector", "Detect and prevent content plagiarism")
            ]
        },
        "technical_operations": {
            bots: [
                create_bot("system_monitor", "Monitor system health and performance"),
                create_bot("security_scanner", "Scan for security vulnerabilities"),
                create_bot("backup_manager", "Manage data backups and recovery"),
                create_bot("update_coordinator", "Coordinate system updates"),
                create_bot("performance_optimizer", "Optimize system performance")
            ]
        },
        "data_analytics": {
            bots: [
                create_bot("usage_analyzer", "Analyze platform usage patterns"),
                create_bot("trend_detector", "Detect emerging trends and patterns"),
                create_bot("report_generator", "Generate analytical reports"),
```

```

        create_bot("prediction_engine", "Make predictive analytics"),
        create_bot("data_validator", "Validate data integrity and accuracy")
    ]
},
"communication_outreach": {
    bots: [
        create_bot("newsletter_manager", "Manage newsletter distribution"),
        create_bot("announcement_coordinator", "Coordinate platform announcements"),
        create_bot("partnership_liaison", "Manage external partnerships"),
        create_bot("media_relations", "Handle media relations and PR"),
        create_bot("crisis_communicator", "Manage crisis communications")
    ]
},
"education_training": {
    bots: [
        create_bot("curriculum_designer", "Design educational curricula"),
        create_bot("assessment_creator", "Create learning assessments"),
        create_bot("progress_tracker", "Track learner progress"),
        create_bot("tutor_assistant", "Provide tutoring assistance"),
        create_bot("certification_manager", "Manage certifications and credentials")
    ]
},
"governance_compliance": {
    bots: [
        create_bot("policy_monitor", "Monitor policy compliance"),
        create_bot("audit_assistant", "Assist with auditing processes"),
        create_bot("legal_researcher", "Research legal requirements"),
        create_bot("compliance_checker", "Check regulatory compliance"),
        create_bot("governance_advisor", "Provide governance recommendations")
    ]
}
}

RETURN divisions

FUNCTION create_bot(bot_name, description):
    // Create individual bot instance

    bot = {
        id: generate_unique_id(),
        name: bot_name,
        description: description,
        status: "INACTIVE",
        capabilities: determine_bot_capabilities(bot_name),
        performance_metrics: initialize_performance_metrics(),
        last_activity: NULL,
        assigned_tasks: [],
        human_oversight_required: FALSE
    }

    RETURN bot

FUNCTION process_bot_task(task):
    // Process task through bot system

```

```

// Step 1: Analyze task requirements
task_analysis = analyze_task_requirements(task)

// Step 2: Select appropriate bot
selected_bot = select_optimal_bot(task_analysis)

// Step 3: Check if human oversight required
IF task_analysis.complexity > HIGH_COMPLEXITY_THRESHOLD:
    queue_for_human_oversight(task, selected_bot)
    RETURN task_queued_for_oversight()

// Step 4: Assign task to bot
assign_task_to_bot(task, selected_bot)

// Step 5: Monitor task execution
monitor_task_execution(task, selected_bot)

// Step 6: Validate results
results = validate_bot_results(task, selected_bot)

// Step 7: Update performance metrics
update_bot_performance_metrics(selected_bot, results)

RETURN results

FUNCTION select_optimal_bot(task_analysis):
    // Select best bot for task based on capabilities and availability

    suitable_bots = []

    // Find bots with required capabilities
    FOR each division IN bot_system.divisions:
        FOR each bot IN division.bots:
            IF bot_has_required_capabilities(bot, task_analysis.required_capabilities):
                AND bot.status EQUALS "AVAILABLE":
                    suitable_bots.append(bot)

    // Rank bots by performance and availability
    ranked_bots = rank_bots_by_performance(suitable_bots, task_analysis)

    // Select top-ranked available bot
    RETURN ranked_bots[0]

FUNCTION monitor_bot_performance():
    // Continuous monitoring of bot system performance

    WHILE system_running:
        FOR each division IN bot_system.divisions:
            FOR each bot IN division.bots:
                // Check bot health
                health_status = check_bot_health(bot)

                // Update performance metrics

```

```

        update_performance_metrics(bot, health_status)

        // Check for anomalies
        IF detect_performance_anomaly(bot):
            flag_for_human_review(bot)

        // Auto-restart if needed
        IF bot.status EQUALS "FAILED":
            attempt_bot_restart(bot)

    wait(MONITORING_INTERVAL)

```

Upgrade Propagation System

```

FUNCTION initiate_global_upgrade(file_list, admin_biometric_data):
    // Initiate worldwide upgrade propagation

    // Step 1: Verify admin authentication
    auth_result = authenticate_admin_biometric("admin", admin_biometric_data)
    IF NOT auth_result.success:
        THROW PermissionError("Biometric authentication failed")

    // Step 2: Generate upgrade ID and version
    upgrade_id = generate_unique_upgrade_id()
    version = generate_version_number()

    // Step 3: Process files for upgrade
    processed_files = []
    FOR each file_path IN file_list:
        file_content = read_file(file_path)
        file_hash = calculate_sha256(file_content)

        processed_file = {
            path: file_path,
            hash: file_hash,
            size: length(file_content),
            content: encode_base64(file_content)
        }
        processed_files.append(processed_file)

    // Step 4: Create admin signature
    admin_signature = create_admin_signature(upgrade_id, processed_files, auth_result.quantum_key)
    biometric_hash = hash_biometric_data(admin_biometric_data)

    // Step 5: Create upgrade package
    upgrade_package = {
        upgrade_id: upgrade_id,
        version: version,
        files: processed_files,
        admin_signature: admin_signature,
        biometric_hash: biometric_hash,
        created_at: timestamp(),
        rollback_data: create_rollback_data()
    }

```

```

}

// Step 6: Create blockchain upgrade token
upgrade_token = create_upgrade_token(
    upgrade_id,
    extract_file_paths(processed_files),
    admin_signature,
    biometric_hash
)

// Step 7: Initiate global propagation
propagation_result = initiate_global_propagation(upgrade_package, upgrade_token)

RETURN {
    upgrade_id: upgrade_id,
    token_id: upgrade_token.token_id,
    propagation_status: propagation_result
}

FUNCTION initiate_global_propagation(upgrade_package, upgrade_token):
    // Propagate upgrade across global mesh network

    // Step 1: Create upgrade torrent
    upgrade_torrent = create_upgrade_torrent(upgrade_package)

    // Step 2: Create propagation message
    propagation_message = {
        type: "GLOBAL_UPGRADE_REQUEST",
        upgrade_id: upgrade_package.upgrade_id,
        upgrade_token: upgrade_token.token_id,
        torrent_hash: upgrade_torrent.hash,
        priority: "CRITICAL",
        timestamp: timestamp(),
        admin_signature: upgrade_package.admin_signature,
        verification_data: {
            biometric_hash: upgrade_package.biometric_hash,
            file_count: length(upgrade_package.files),
            total_size: calculate_total_size(upgrade_package.files)
        }
    }

    // Step 3: Broadcast to all mesh nodes
    broadcast_result = broadcast_to_all_nodes(propagation_message)

    // Step 4: Start propagation monitoring
    start_propagation_monitoring(upgrade_package.upgrade_id)

    RETURN broadcast_result

FUNCTION broadcast_to_all_nodes(message):
    // Broadcast message to all active mesh nodes

    active_nodes = get_active_mesh_nodes()
    broadcast_results = {}

```

```

// Send message to each node in parallel
FOR each node IN active_nodes:
    ASYNC:
        try:
            result = send_message_to_node(node, message)
            broadcast_results[node.id] = result
        catch NetworkError:
            broadcast_results[node.id] = {success: FALSE, error: "Network timeout"}

// Wait for all broadcasts to complete
wait_for_all_async_operations()

// Calculate success rate
successful_broadcasts = count_successful_broadcasts(broadcast_results)
total_nodes = length(active_nodes)
success_rate = successful_broadcasts / total_nodes

RETURN {
    total_nodes: total_nodes,
    successful_broadcasts: successful_broadcasts,
    success_rate: success_rate,
    detailed_results: broadcast_results
}

FUNCTION monitor_propagation_progress(upgrade_id):
    // Monitor upgrade propagation across network

    start_time = timestamp()
    timeout = PROPAGATION_TIMEOUT

    WHILE (timestamp() - start_time) < timeout:
        // Check propagation status
        progress = check_propagation_progress(upgrade_id)

        // Log progress
        log_propagation_progress(upgrade_id, progress)

        // Check completion threshold
        IF progress.completion_percentage >= COMPLETION_THRESHOLD:
            finalize_global_upgrade(upgrade_id)
            RETURN propagation_success(progress)

        // Check for failures
        IF progress.failed_nodes > FAILURE_THRESHOLD:
            handle_propagation_failures(upgrade_id, progress)

        wait(PROGRESS_CHECK_INTERVAL)

    // Timeout reached
    RETURN propagation_timeout(upgrade_id)

FUNCTION check_propagation_progress(upgrade_id):
    // Check upgrade progress across all nodes

```

```

active_nodes = get_active_mesh_nodes()
progress_data = {
    total_nodes: length(active_nodes),
    completed_nodes: 0,
    failed_nodes: 0,
    in_progress_nodes: 0,
    node_statuses: {}
}

FOR each node IN active_nodes:
    try:
        status = query_node_upgrade_status(node, upgrade_id)
        progress_data.node_statuses[node.id] = status

        SWITCH status.state:
            CASE "COMPLETED":
                progress_data.completed_nodes += 1
            CASE "FAILED":
                progress_data.failed_nodes += 1
            CASE "IN_PROGRESS":
                progress_data.in_progress_nodes += 1
    catch NetworkError:
        progress_data.failed_nodes += 1
        progress_data.node_statuses[node.id] = {state: "UNREACHABLE"}

// Calculate completion percentage
progress_data.completion_percentage = (progress_data.completed_nodes / progress_data.total_nodes) *

RETURN progress_data

FUNCTION handle_node_upgrade_request(upgrade_request):
    // Handle incoming upgrade request at mesh node

    // Step 1: Verify upgrade token
    token_valid = verify_upgrade_token(upgrade_request.upgrade_token)
    IF NOT token_valid:
        RETURN upgrade_rejected("Invalid upgrade token")

    // Step 2: Verify admin signature
    signature_valid = verify_admin_signature(
        upgrade_request.admin_signature,
        upgrade_request.upgrade_id
    )
    IF NOT signature_valid:
        RETURN upgrade_rejected("Invalid admin signature")

    // Step 3: Download upgrade package
    upgrade_package = download_upgrade_torrent(upgrade_request.torrent_hash)
    IF upgrade_package IS NULL:
        RETURN upgrade_failed("Failed to download upgrade package")

    // Step 4: Verify package integrity
    package_valid = verify_package_integrity(upgrade_package)

```



```

IF NOT package_valid:
    RETURN upgrade_failed("Package integrity verification failed")

// Step 5: Create backup
backup_result = create_system_backup()
IF NOT backup_result.success:
    RETURN upgrade_failed("Failed to create system backup")

// Step 6: Apply upgrade
upgrade_result = apply_upgrade_package(upgrade_package)
IF NOT upgrade_result.success:
    // Rollback on failure
    rollback_result = rollback_to_backup(backup_result.backup_id)
    RETURN upgrade_failed("Upgrade failed, rolled back to previous state")

// Step 7: Verify upgrade success
verification_result = verify_upgrade_success(upgrade_package)
IF NOT verification_result.success:
    rollback_result = rollback_to_backup(backup_result.backup_id)
    RETURN upgrade_failed("Upgrade verification failed, rolled back")

// Step 8: Update local version
update_local_version(upgrade_package.version)

// Step 9: Report success
report_upgrade_success(upgrade_request.upgrade_id)

RETURN upgrade_success("Upgrade completed successfully")

```

Frontend Application

```

FUNCTION initialize_frontend_application():
    // Initialize React.js PWA frontend

    app_config = {
        api_base_url: get_env("REACT_APP_API_URL"),
        websocket_url: get_env("REACT_APP_WS_URL"),
        supported_languages: load_supported_languages(),
        theme_config: load_theme_configuration(),
        pwa_config: load_pwa_configuration()
    }

    // Initialize core services
    auth_service = initialize_auth_service()
    api_service = initialize_api_service(app_config.api_base_url)
    websocket_service = initialize_websocket_service(app_config.websocket_url)
    i18n_service = initialize_internationalization_service()

    // Initialize state management
    redux_store = initialize_redux_store()

    // Initialize routing
    router = initialize_react_router()

```

```

RETURN app_instance

FUNCTION render_main_application():
    // Main application component

    COMPONENT MainApplication:
        // State management
        user_state = useSelector(selectUser)
        system_state = useSelector(selectSystemStatus)

        // Effects
        useEffect(() => {
            // Initialize authentication check
            check_authentication_status()

            // Start system monitoring
            start_system_status_monitoring()

            // Initialize websocket connection
            establish_websocket_connection()
        }, [])

        // Render application
        RETURN (
            <Router>
                <ThemeProvider theme={app_theme}>
                    <CssBaseline />
                    <ErrorBoundary>
                        <AuthenticationProvider>
                            <NotificationProvider>
                                <Routes>
                                    <Route path="/" element={<LandingPage />} />
                                    <Route path="/login" element={<BiometricLoginPage />} />
                                    <Route path="/dashboard" element={<AdminDashboard />} />
                                    <Route path="/system" element={<SystemMonitoring />} />
                                    <Route path="/bots" element={<BotManagement />} />
                                    <Route path="/blockchain" element={<BlockchainViewer />} />
                                    <Route path="/distribution" element={<DistributionStatus />} />
                                    <Route path="/upgrades" element={<UpgradeManager />} />
                                </Routes>
                            </NotificationProvider>
                        </AuthenticationProvider>
                    </ErrorBoundary>
                </ThemeProvider>
            </Router>
        )

    )

FUNCTION BiometricLoginPage():
    // Biometric authentication interface

    COMPONENT BiometricLoginPage:
        // State
        const [authStep, setAuthStep] = useState("INITIAL")

```

```

const [biometricData, setBiometricData] = useState({})
const [authProgress, setAuthProgress] = useState(0)

// Biometric capture functions
const captureFingerprint = async () => {
  const fingerprintData = await biometric_scanner.captureFingerprint()
  setBiometricData(prev => ({...prev, fingerprint: fingerprintData}))
  setAuthProgress(prev => prev + 20)
}

const captureIris = async () => {
  const irisData = await biometric_scanner.captureIris()
  setBiometricData(prev => ({...prev, iris: irisData}))
  setAuthProgress(prev => prev + 20)
}

const captureVoice = async () => {
  const voiceData = await biometric_scanner.captureVoice()
  setBiometricData(prev => ({...prev, voice: voiceData}))
  setAuthProgress(prev => prev + 20)
}

const captureFace = async () => {
  const faceData = await biometric_scanner.captureFace()
  setBiometricData(prev => ({...prev, face: faceData}))
  setAuthProgress(prev => prev + 20)
}

const captureBehavioral = async () => {
  const behavioralData = await biometric_scanner.captureBehavioral()
  setBiometricData(prev => ({...prev, behavioral: behavioralData}))
  setAuthProgress(prev => prev + 20)
}

const performAuthentication = async () => {
  setAuthStep("AUTHENTICATING")

  try:
    const authResult = await auth_service.authenticateWithBiometrics(
      "admin",
      biometricData
    )

    IF authResult.success:
      // Store authentication tokens
      store_auth_tokens(authResult.session_token, authResult.quantum_key)

      // Redirect to dashboard
      navigate("/dashboard")
    ELSE:
      setAuthStep("FAILED")
      show_error_message("Biometric authentication failed")
  } catch (error):
    setAuthStep("FAILED")

```

```

        show_error_message("Authentication error: " + error.message)
    }

    // Render authentication interface
    RETURN (
        <Container maxWidth="md">
            <Paper elevation={3} sx={{p: 4}}>
                <Typography variant="h4" align="center" gutterBottom>
                    Biometric Administration Access
                </Typography>

                <LinearProgress variant="determinate" value={authProgress} />

                <Grid container spacing={3}>
                    <Grid item xs={12} md={6}>
                        <BiometricCaptureCard
                            title="Fingerprint"
                            icon={<FingerprintIcon />}
                            onCapture={captureFingerprint}
                            captured={biometricData.fingerprint != null}
                        />
                    </Grid>

                    <Grid item xs={12} md={6}>
                        <BiometricCaptureCard
                            title="Iris Scan"
                            icon={<VisibilityIcon />}
                            onCapture={captureIris}
                            captured={biometricData.iris != null}
                        />
                    </Grid>

                    <Grid item xs={12} md={6}>
                        <BiometricCaptureCard
                            title="Voice Recognition"
                            icon={<MicIcon />}
                            onCapture={captureVoice}
                            captured={biometricData.voice != null}
                        />
                    </Grid>

                    <Grid item xs={12} md={6}>
                        <BiometricCaptureCard
                            title="Facial Recognition"
                            icon={<FaceIcon />}
                            onCapture={captureFace}
                            captured={biometricData.face != null}
                        />
                    </Grid>

                    <Grid item xs={12}>
                        <BiometricCaptureCard
                            title="Behavioral Analysis"
                            icon={<PsychologyIcon />}

```

```

        onCapture={captureBehavioral}
        captured={biometricData.behavioral != null}
      />
    </Grid>
  </Grid>

  <Box sx={{mt: 4, textAlign: "center"}}>
    <Button
      variant="contained"
      size="large"
      onClick={performAuthentication}
      disabled={authProgress < 60 || authStep === "AUTHENTICATING"}
      startIcon={<SecurityIcon />}
    >
      {authStep === "AUTHENTICATING" ? "Authenticating..." : "Authenticate"}
    </Button>
  </Box>
</Paper>
</Container>
)

```

FUNCTION AdminDashboard():

// Main administration dashboard

COMPONENT AdminDashboard:

// State

const [systemStatus, setSystemStatus] = useState(null)

const [networkHealth, setNetworkHealth] = useState(null)

const [botStatus, setBotStatus] = useState(null)

const [upgradeStatus, setUpgradeStatus] = useState(null)

// Effects

useEffect(() => {

// Load dashboard data

loadDashboardData()

// Set up real-time updates

const interval = setInterval(loadDashboardData, 30000)

return () => clearInterval(interval)

}, [])

const loadDashboardData = async () => {

try:

```

    const [system, network, bots, upgrades] = await Promise.all([
      api_service.getSystemStatus(),
      api_service.getNetworkHealth(),
      api_service.getBotStatus(),
      api_service.getUpgradeStatus()
    ])

```

setSystemStatus(system)

setNetworkHealth(network)

setBotStatus(bots)

```

        setUpgradeStatus(upgrades)
    } catch (error):
        show_error_message("Failed to load dashboard data")
}

// Render dashboard
RETURN (
    <Container maxWidth="xl">
        <Typography variant="h3" gutterBottom>
            International Plebeian Academy - Administration
        </Typography>

        <Grid container spacing={3}>
            // System Status Cards
            <Grid item xs={12} md={3}>
                <StatusCard
                    title="System Health"
                    value={systemStatus?.health_percentage || 0}
                    icon={<HealthAndSafetyIcon />}
                    color="primary"
                />
            </Grid>

            <Grid item xs={12} md={3}>
                <StatusCard
                    title="Network Nodes"
                    value={networkHealth?.active_nodes || 0}
                    subtitle={`of ${networkHealth?.total_nodes || 0} total`}
                    icon={<NetworkCheckIcon />}
                    color="secondary"
                />
            </Grid>

            <Grid item xs={12} md={3}>
                <StatusCard
                    title="Active Bots"
                    value={botStatus?.active_bots || 0}
                    subtitle={`across ${botStatus?.divisions || 0} divisions`}
                    icon={<SmartToyIcon />}
                    color="success"
                />
            </Grid>

            <Grid item xs={12} md={3}>
                <StatusCard
                    title="Pending Upgrades"
                    value={upgradeStatus?.pending_upgrades || 0}
                    icon={<SystemUpdateIcon />}
                    color="warning"
                />
            </Grid>

            // Real-time Charts
            <Grid item xs={12} md={8}>

```

```

        <Paper sx={{p: 2}}>
          <Typography variant="h6" gutterBottom>
            Network Activity
          </Typography>
          <NetworkActivityChart data={networkHealth?.activity_data} />
        </Paper>
      </Grid>

      <Grid item xs={12} md={4}>
        <Paper sx={{p: 2}}>
          <Typography variant="h6" gutterBottom>
            Bot Performance
          </Typography>
          <BotPerformanceChart data={botStatus?.performance_data} />
        </Paper>
      </Grid>

      // Quick Actions
      <Grid item xs={12}>
        <Paper sx={{p: 2}}>
          <Typography variant="h6" gutterBottom>
            Quick Actions
          </Typography>
          <Grid container spacing={2}>
            <Grid item>
              <Button
                variant="contained"
                startIcon={<SystemUpdateIcon />}
                onClick={() => navigate("/upgrades")}
              >
                Initiate Upgrade
              </Button>
            </Grid>
            <Grid item>
              <Button
                variant="outlined"
                startIcon={<SmartToyIcon />}
                onClick={() => navigate("/bots")}
              >
                Manage Bots
              </Button>
            </Grid>
            <Grid item>
              <Button
                variant="outlined"
                startIcon={<SecurityIcon />}
                onClick={() => navigate("/blockchain")}
              >
                Blockchain Status
              </Button>
            </Grid>
          </Grid>
        </Paper>
      </Grid>

```

```

        </Grid>
    </Container>
)

```

Backend API System

```

FUNCTION initialize_backend_application():
    // Initialize Flask backend with Seven Division architecture

    app = Flask(__name__)

    // Configure application
    app.config.from_object(Config)

    // Initialize extensions
    db = initialize_database(app)
    redis_client = initialize_redis(app)
    socketio = initialize_websocket(app)

    // Initialize core services
    auth_service = initialize_auth_service()
    biometric_service = initialize_biometric_service()
    blockchain_service = initialize_blockchain_service()
    distribution_service = initialize_distribution_service()
    bot_service = initialize_bot_service()
    upgrade_service = initialize_upgrade_service()

    // Register blueprints
    register_api_blueprints(app)

    // Set up middleware
    setup_security_middleware(app)
    setup_cors_middleware(app)
    setup_rate_limiting(app)

    RETURN app

FUNCTION register_api_blueprints(app):
    // Register all API route blueprints

    // Authentication routes
    app.register_blueprint(auth_blueprint, url_prefix="/api/auth")

    // System management routes
    app.register_blueprint(system_blueprint, url_prefix="/api/system")

    // Blockchain routes
    app.register_blueprint(blockchain_blueprint, url_prefix="/api/blockchain")

    // Distribution routes
    app.register_blueprint(distribution_blueprint, url_prefix="/api/distribution")

    // Bot management routes

```



```

app.register_blueprint(bot_blueprint, url_prefix="/api/bots")

// Upgrade management routes
app.register_blueprint(upgrade_blueprint, url_prefix="/api/upgrades")

// File verification routes
app.register_blueprint(file_blueprint, url_prefix="/api/files")

// Authentication API Routes
BLUEPRINT auth_blueprint:

    @route("/login", methods=["POST"])
    FUNCTION biometric_login():
        // Handle biometric authentication

        request_data = get_json_from_request()
        user_id = request_data.get("user_id")
        biometric_data = request_data.get("biometric_data")

        // Validate input
        IF NOT user_id OR NOT biometric_data:
            RETURN error_response("Missing required fields", 400)

        try:
            // Perform biometric authentication
            auth_result = biometric_service.authenticate_user(user_id, biometric_data)

            IF auth_result.success:
                // Create session
                session_data = {
                    "user_id": user_id,
                    "session_token": auth_result.session_token,
                    "quantum_key": auth_result.quantum_key,
                    "expires_at": auth_result.expires_at,
                    "confidence_score": auth_result.confidence_score
                }

                // Store session in Redis
                redis_client.setex(
                    f"session:{auth_result.session_token}",
                    SESSION_DURATION,
                    json.dumps(session_data)
                )

                RETURN success_response({
                    "session_token": auth_result.session_token,
                    "expires_at": auth_result.expires_at,
                    "confidence_score": auth_result.confidence_score
                })
            ELSE:
                RETURN error_response("Authentication failed", 401)

        except Exception as e:
            log_error("Biometric authentication error", e)

```

```

        RETURN error_response("Authentication error", 500)

@route("/logout", methods=["POST"])
@require_authentication
FUNCTION logout():
    // Handle user logout

    session_token = get_session_token_from_request()

    // Remove session from Redis
    redis_client.delete(f"session:{session_token}")

    RETURN success_response({"message": "Logged out successfully"})

@route("/verify", methods=["GET"])
@require_authentication
FUNCTION verify_session():
    // Verify current session

    session_data = get_current_session()

    RETURN success_response({
        "valid": TRUE,
        "user_id": session_data["user_id"],
        "expires_at": session_data["expires_at"]
    })

// System Management API Routes
BLUEPRINT system_blueprint:

@route("/status", methods=["GET"])
@require_authentication
FUNCTION get_system_status():
    // Get comprehensive system status

    try:
        system_status = {
            "health_percentage": calculate_system_health(),
            "uptime": get_system_uptime(),
            "version": get_current_version(),
            "last_upgrade": get_last_upgrade_info(),
            "active_services": get_active_services(),
            "resource_usage": get_resource_usage(),
            "security_status": get_security_status()
        }

        RETURN success_response(system_status)

    except Exception as e:
        log_error("System status error", e)
        RETURN error_response("Failed to get system status", 500)

@route("/health", methods=["GET"])
FUNCTION health_check():

```

```

// Simple health check endpoint

health_data = {
    "status": "healthy",
    "timestamp": timestamp(),
    "version": get_current_version(),
    "services": {
        "database": check_database_health(),
        "redis": check_redis_health(),
        "blockchain": check_blockchain_health(),
        "distribution": check_distribution_health()
    }
}

RETURN success_response(health_data)

@route("/metrics", methods=["GET"])
@require_authentication
FUNCTION get_system_metrics():
    // Get detailed system metrics

    try:
        metrics = {
            "performance": get_performance_metrics(),
            "network": get_network_metrics(),
            "blockchain": get_blockchain_metrics(),
            "distribution": get_distribution_metrics(),
            "bots": get_bot_metrics(),
            "security": get_security_metrics()
        }

        RETURN success_response(metrics)

    except Exception as e:
        log_error("Metrics error", e)
        RETURN error_response("Failed to get metrics", 500)

// Upgrade Management API Routes
BLUEPRINT upgrade_blueprint:

@route("/initiate", methods=["POST"])
@require_admin_authentication
FUNCTION initiate_upgrade():
    // Initiate global upgrade

    request_data = get_json_from_request()
    file_list = request_data.get("files", [])
    biometric_data = request_data.get("biometric_data")

    // Validate input
    IF NOT file_list OR NOT biometric_data:
        RETURN error_response("Missing required fields", 400)

    try:

```

```

        // Initiate upgrade
        upgrade_result = upgrade_service.initiate_global_upgrade(
            file_list,
            biometric_data
        )

        RETURN success_response({
            "upgrade_id": upgrade_result.upgrade_id,
            "token_id": upgrade_result.token_id,
            "status": "initiated"
        })

    except PermissionError as e:
        RETURN error_response("Authentication failed", 401)
    except Exception as e:
        log_error("Upgrade initiation error", e)
        RETURN error_response("Failed to initiate upgrade", 500)

@route("/status/<upgrade_id>", methods=["GET"])
@require_authentication
FUNCTION get_upgrade_status(upgrade_id):
    // Get upgrade status

    try:
        status = upgrade_service.get_upgrade_status(upgrade_id)

        IF status IS NULL:
            RETURN error_response("Upgrade not found", 404)

        RETURN success_response(status)

    except Exception as e:
        log_error("Upgrade status error", e)
        RETURN error_response("Failed to get upgrade status", 500)

@route("/history", methods=["GET"])
@require_authentication
FUNCTION get_upgrade_history():
    // Get upgrade history

    try:
        history = upgrade_service.get_upgrade_history()
        RETURN success_response(history)

    except Exception as e:
        log_error("Upgrade history error", e)
        RETURN error_response("Failed to get upgrade history", 500)

// WebSocket Events
SOCKETIO_NAMESPACE "/system":

    @event("connect")
    FUNCTION on_connect():
        // Handle client connection

```

```

    session_token = get_session_token_from_auth()
    IF NOT validate_session_token(session_token):
        disconnect()
        RETURN

    join_room("system_updates")
    emit("connected", {"status": "connected"})

@event("disconnect")
FUNCTION on_disconnect():
    // Handle client disconnection

    leave_room("system_updates")

@event("subscribe_to_upgrades")
FUNCTION on_subscribe_upgrades():
    // Subscribe to upgrade notifications

    join_room("upgrade_updates")
    emit("subscribed", {"channel": "upgrades"})

FUNCTION broadcast_system_update(update_type, data):
    // Broadcast system updates to connected clients

    socketio.emit("system_update", {
        "type": update_type,
        "data": data,
        "timestamp": timestamp()
    }, room="system_updates")

FUNCTION broadcast_upgrade_update(upgrade_id, status_data):
    // Broadcast upgrade updates

    socketio.emit("upgrade_update", {
        "upgrade_id": upgrade_id,
        "status": status_data,
        "timestamp": timestamp()
    }, room="upgrade_updates")

```

Tribal Coin Blockchain

```

FUNCTION initialize_tribal_coin_system():
    // Initialize Tribal Coin blockchain and economic system

    // Initialize Web3 connection
    web3_provider = initialize_web3_provider()

    // Load smart contracts
    tribal_coin_contract = load_contract(TRIBAL_COIN_CONTRACT_ADDRESS)
    governance_contract = load_contract(GOVERNANCE_CONTRACT_ADDRESS)

    // Initialize economic systems

```

```

reward_system = initialize_economic_reward_system()
mutual_aid_system = initialize_mutual_aid_system()

// Initialize governance
governance_system = initialize_governance_system()

RETURN {
    web3: web3_provider,
    contracts: {
        tribal_coin: tribal_coin_contract,
        governance: governance_contract
    },
    economic: {
        rewards: reward_system,
        mutual_aid: mutual_aid_system
    },
    governance: governance_system
}

FUNCTION deploy_tribal_coin_contract():
    // Deploy Tribal Coin smart contract

    contract_source = ""
    pragma solidity ^0.8.0;

    contract TribalCoin {
        string public name = "Tribal Coin";
        string public symbol = "TC";
        uint8 public decimals = 18;
        uint256 public totalSupply;

        mapping(address => uint256) public balanceOf;
        mapping(address => mapping(address => uint256)) public allowance;

        // Governance structures
        struct Proposal {
            uint256 id;
            string description;
            uint256 votesFor;
            uint256 votesAgainst;
            uint256 deadline;
            bool executed;
            mapping(address => bool) hasVoted;
        }

        mapping(uint256 => Proposal) public proposals;
        uint256 public proposalCount;

        // Events
        event Transfer(address indexed from, address indexed to, uint256 value);
        event Approval(address indexed owner, address indexed spender, uint256 value);
        event ProposalCreated(uint256 indexed proposalId, string description);
        event VoteCast(uint256 indexed proposalId, address indexed voter, bool support);

```

```

constructor(uint256 _initialSupply) {
    totalSupply = _initialSupply * 10**decimals;
    balanceOf[msg.sender] = totalSupply;
}

function transfer(address _to, uint256 _value) public returns (bool) {
    require(balanceOf[msg.sender] >= _value, "Insufficient balance");
    balanceOf[msg.sender] -= _value;
    balanceOf[_to] += _value;
    emit Transfer(msg.sender, _to, _value);
    return true;
}

function approve(address _spender, uint256 _value) public returns (bool) {
    allowance[msg.sender][_spender] = _value;
    emit Approval(msg.sender, _spender, _value);
    return true;
}

function transferFrom(address _from, address _to, uint256 _value) public returns (bool) {
    require(balanceOf[_from] >= _value, "Insufficient balance");
    require(allowance[_from][msg.sender] >= _value, "Insufficient allowance");

    balanceOf[_from] -= _value;
    balanceOf[_to] += _value;
    allowance[_from][msg.sender] -= _value;

    emit Transfer(_from, _to, _value);
    return true;
}

function mint(address _to, uint256 _value) public {
    // Only authorized minters can mint
    require(authorizedMinters[msg.sender], "Not authorized to mint");

    totalSupply += _value;
    balanceOf[_to] += _value;
    emit Transfer(address(0), _to, _value);
}

function createProposal(string memory _description) public returns (uint256) {
    require(balanceOf[msg.sender] >= PROPOSAL_THRESHOLD, "Insufficient tokens to create proposal");

    proposalCount++;
    Proposal storage newProposal = proposals[proposalCount];
    newProposal.id = proposalCount;
    newProposal.description = _description;
    newProposal.deadline = block.timestamp + VOTING_PERIOD;

    emit ProposalCreated(proposalCount, _description);
    return proposalCount;
}

function vote(uint256 _proposalId, bool _support) public {

```

```

        Proposal storage proposal = proposals[_proposalId];
        require(block.timestamp < proposal.deadline, "Voting period ended");
        require(!proposal.hasVoted[msg.sender], "Already voted");
        require(balanceOf[msg.sender] > 0, "No voting power");

        proposal.hasVoted[msg.sender] = true;
        uint256 votingPower = balanceOf[msg.sender];

        if (_support) {
            proposal.votesFor += votingPower;
        } else {
            proposal.votesAgainst += votingPower;
        }

        emit VoteCast(_proposalId, msg.sender, _support);
    }
}
"""

// Compile and deploy contract
compiled_contract = compile_solidity(contract_source)
deployment_transaction = web3.eth.contract(
    abi=compiled_contract.abi,
    bytecode=compiled_contract.bytecode
).constructor(INITIAL_SUPPLY).buildTransaction()

// Sign and send transaction
signed_transaction = web3.eth.account.sign_transaction(
    deployment_transaction,
    private_key=ADMIN_PRIVATE_KEY
)

transaction_hash = web3.eth.send_raw_transaction(signed_transaction.rawTransaction)

// Wait for deployment
receipt = web3.eth.wait_for_transaction_receipt(transaction_hash)

RETURN receipt.contractAddress

FUNCTION calculate_economic_reward(user_address, activity_type, quality_score, quantity):
    // Calculate reward based on activity and quality

    base_rewards = {
        "content_creation": 10.0,
        "community_moderation": 5.0,
        "educational_contribution": 15.0,
        "platform_development": 20.0,
        "governance_participation": 8.0,
        "mutual_aid_contribution": 12.0,
        "knowledge_sharing": 7.0
    }

    base_reward = base_rewards.get(activity_type, 1.0)
    quality_multiplier = max(0.1, min(2.0, quality_score))

```



```

quantity_multiplier = quantity

// Apply user reputation multiplier
user_reputation = get_user_reputation(user_address)
reputation_multiplier = 1.0 + (user_reputation / 100.0)

// Apply daily earning limits
daily_earnings = get_daily_earnings(user_address)
daily_limit = DAILY_EARNING_LIMIT

IF daily_earnings >= daily_limit:
    RETURN 0.0 // Daily limit reached

total_reward = base_reward * quality_multiplier * quantity_multiplier * reputation_multiplier

// Ensure doesn't exceed daily limit
total_reward = min(total_reward, daily_limit - daily_earnings)

RETURN total_reward

FUNCTION award_tribal_coins(user_address, amount, activity_type):
    // Award Tribal Coins to user

    try:
        // Create mint transaction
        mint_transaction = tribal_coin_contract.functions.mint(
            user_address,
            web3.toWei(amount, 'ether')
        ).buildTransaction({
            'from': ADMIN_ADDRESS,
            'gas': 100000,
            'gasPrice': web3.toWei('20', 'gwei'),
            'nonce': web3.eth.get_transaction_count(ADMIN_ADDRESS)
        })

        // Sign transaction
        signed_transaction = web3.eth.account.sign_transaction(
            mint_transaction,
            private_key=ADMIN_PRIVATE_KEY
        )

        // Send transaction
        transaction_hash = web3.eth.send_raw_transaction(signed_transaction.rawTransaction)

        // Wait for confirmation
        receipt = web3.eth.wait_for_transaction_receipt(transaction_hash)

        // Update user records
        update_user_earnings(user_address, amount, activity_type)

    RETURN {
        "success": TRUE,
        "transaction_hash": transaction_hash.hex(),
        "amount": amount,

```

```

        "new_balance": get_user_balance(user_address)
    }

except Exception as e:
    log_error("Token award error", e)
    RETURN {
        "success": FALSE,
        "error": str(e)
    }

FUNCTION create_governance_proposal(proposer_address, description, proposal_data):
    // Create governance proposal

    // Check proposer has sufficient tokens
    proposer_balance = get_user_balance(proposer_address)
    IF proposer_balance < PROPOSAL_THRESHOLD:
        RETURN proposal_failed("Insufficient tokens to create proposal")

    try:
        // Create proposal transaction
        proposal_transaction = governance_contract.functions.createProposal(
            description,
            json.dumps(proposal_data)
        ).buildTransaction({
            'from': proposer_address,
            'gas': 200000,
            'gasPrice': web3.toWei('20', 'gwei'),
            'nonce': web3.eth.get_transaction_count(proposer_address)
        })

        // This would be signed by the proposer's wallet
        # For demo purposes, we'll simulate the process

        proposal_id = get_next_proposal_id()

        proposal_record = {
            "id": proposal_id,
            "proposer": proposer_address,
            "description": description,
            "data": proposal_data,
            "created_at": timestamp(),
            "voting_deadline": timestamp() + VOTING_PERIOD,
            "votes_for": 0,
            "votes_against": 0,
            "status": "ACTIVE"
        }

        store_proposal(proposal_record)

    RETURN {
        "success": TRUE,
        "proposal_id": proposal_id,
        "voting_deadline": proposal_record["voting_deadline"]
    }

```

```

except Exception as e:
    log_error("Proposal creation error", e)
    RETURN proposal_failed("Failed to create proposal")

FUNCTION vote_on_proposal(voter_address, proposal_id, support):
    // Vote on governance proposal

    proposal = get_proposal(proposal_id)
    IF proposal IS NULL:
        RETURN vote_failed("Proposal not found")

    IF timestamp() > proposal["voting_deadline"]:
        RETURN vote_failed("Voting period ended")

    IF has_voted(voter_address, proposal_id):
        RETURN vote_failed("Already voted on this proposal")

    voter_balance = get_user_balance(voter_address)
    IF voter_balance <= 0:
        RETURN vote_failed("No voting power")

    try:
        // Record vote
        vote_record = {
            "proposal_id": proposal_id,
            "voter": voter_address,
            "support": support,
            "voting_power": voter_balance,
            "timestamp": timestamp()
        }

        store_vote(vote_record)

        // Update proposal vote counts
        IF support:
            proposal["votes_for"] += voter_balance
        ELSE:
            proposal["votes_against"] += voter_balance

        update_proposal(proposal)

        RETURN {
            "success": TRUE,
            "vote_recorded": TRUE,
            "voting_power": voter_balance
        }

    except Exception as e:
        log_error("Voting error", e)
        RETURN vote_failed("Failed to record vote")

FUNCTION process_mutual_aid_request(requester_address, amount_needed, description, category):
    // Process mutual aid request

```

```

// Validate request
IF amount_needed <= 0 OR amount_needed > MAX_AID_REQUEST:
    RETURN aid_request_failed("Invalid amount")

// Check user eligibility
user_reputation = get_user_reputation(requester_address)
IF user_reputation < MINIMUM_AID_REPUTATION:
    RETURN aid_request_failed("Insufficient reputation for aid request")

// Create aid request
aid_request = {
    "id": generate_unique_id(),
    "requester": requester_address,
    "amount_needed": amount_needed,
    "amount_raised": 0,
    "description": description,
    "category": category,
    "created_at": timestamp(),
    "deadline": timestamp() + AID_REQUEST_DURATION,
    "status": "ACTIVE",
    "contributors": []
}

store_aid_request(aid_request)

// Notify community
broadcast_aid_request(aid_request)

RETURN {
    "success": TRUE,
    "request_id": aid_request["id"],
    "deadline": aid_request["deadline"]
}

}

FUNCTION contribute_to_aid_request(contributor_address, request_id, contribution_amount):
    // Contribute to mutual aid request

    aid_request = get_aid_request(request_id)
    IF aid_request IS NULL:
        RETURN contribution_failed("Aid request not found")

    IF aid_request["status"] != "ACTIVE":
        RETURN contribution_failed("Aid request not active")

    IF timestamp() > aid_request["deadline"]:
        RETURN contribution_failed("Aid request expired")

    // Check contributor balance
    contributor_balance = get_user_balance(contributor_address)
    IF contributor_balance < contribution_amount:
        RETURN contribution_failed("Insufficient balance")

try:

```

```

// Transfer tokens to aid pool
transfer_result = transfer_tokens(
    contributor_address,
    AID_POOL_ADDRESS,
    contribution_amount
)

IF NOT transfer_result.success:
    RETURN contribution_failed("Transfer failed")

// Record contribution
contribution = {
    "contributor": contributor_address,
    "amount": contribution_amount,
    "timestamp": timestamp()
}

aid_request["contributors"].append(contribution)
aid_request["amount_raised"] += contribution_amount

// Check if goal reached
IF aid_request["amount_raised"] >= aid_request["amount_needed"]:
    aid_request["status"] = "FUNDED"

    // Transfer funds to requester
    transfer_tokens(
        AID_POOL_ADDRESS,
        aid_request["requester"],
        aid_request["amount_needed"]
    )

    // Return excess to contributors proportionally
    IF aid_request["amount_raised"] > aid_request["amount_needed"]:
        distribute_excess_funds(aid_request)

update_aid_request(aid_request)

RETURN {
    "success": TRUE,
    "contribution_amount": contribution_amount,
    "total_raised": aid_request["amount_raised"],
    "goal_reached": aid_request["status"] == "FUNDED"
}

except Exception as e:
    log_error("Aid contribution error", e)
    RETURN contribution_failed("Failed to process contribution")

```

Infrastructure & Deployment

```

FUNCTION initialize_deployment_infrastructure():
    // Initialize comprehensive deployment and monitoring infrastructure

```

```

deployment_config = {
    "environments": ["development", "staging", "production"],
    "cloud_provider": "kubernetes",
    "container_registry": "docker_hub",
    "monitoring_stack": ["prometheus", "grafana", "alertmanager"],
    "logging_stack": ["elasticsearch", "logstash", "kibana"],
    "backup_strategy": "automated_daily",
    "security_scanning": "enabled",
    "auto_scaling": "enabled"
}

// Initialize deployment managers
kubernetes_manager = initialize_kubernetes_deployment()
docker_manager = initialize_docker_deployment()
monitoring_manager = initialize_monitoring_system()

RETURN {
    "config": deployment_config,
    "managers": {
        "kubernetes": kubernetes_manager,
        "docker": docker_manager,
        "monitoring": monitoring_manager
    }
}

FUNCTION deploy_to_production():
    // Deploy complete system to production environment

    // Step 1: Pre-deployment checks
    pre_deployment_checks = run_pre_deployment_checks()
    IF NOT pre_deployment_checks.all_passed:
        RETURN deployment_failed("Pre-deployment checks failed")

    // Step 2: Create deployment manifests
    manifests = generate_kubernetes_manifests("production")

    // Step 3: Deploy core services
    core_services = [
        "postgresql-database",
        "redis-cache",
        "ganache-blockchain",
        "backend-api",
        "frontend-app",
        "nginx-proxy"
    ]

    FOR each service IN core_services:
        deployment_result = deploy_service(service, "production")
        IF NOT deployment_result.success:
            rollback_deployment()
            RETURN deployment_failed(f"Failed to deploy {service}")

    // Step 4: Deploy specialized services
    specialized_services = [

```

```

        "biometric-service",
        "quantum-security-service",
        "holographic-distributor",
        "torrent-coordinator",
        "bot-orchestrator",
        "upgrade-manager"
    ]

    FOR each service IN specialized_services:
        deployment_result = deploy_service(service, "production")
        IF NOT deployment_result.success:
            log_warning(f"Specialized service {service} deployment failed")
            // Continue with other services

    // Step 5: Configure networking
    networking_result = configure_production_networking()
    IF NOT networking_result.success:
        RETURN deployment_failed("Network configuration failed")

    // Step 6: Set up monitoring
    monitoring_result = setup_production_monitoring()
    IF NOT monitoring_result.success:
        log_warning("Monitoring setup failed")

    // Step 7: Configure backups
    backup_result = configure_automated_backups()
    IF NOT backup_result.success:
        log_warning("Backup configuration failed")

    // Step 8: Run post-deployment tests
    post_deployment_tests = run_post_deployment_tests()
    IF NOT post_deployment_tests.all_passed:
        log_warning("Some post-deployment tests failed")

    // Step 9: Update DNS and SSL
    dns_result = update_production_dns()
    ssl_result = configure_ssl_certificates()

    RETURN deployment_success("Production deployment completed")

FUNCTION generate_kubernetes_manifests(environment):
    // Generate Kubernetes deployment manifests

    manifests = {}

    // Database deployment
    manifests["postgresql"] = {
        "apiVersion": "apps/v1",
        "kind": "Deployment",
        "metadata": {
            "name": "postgresql-deployment",
            "namespace": "plebeian-academy"
        },
        "spec": {

```

```

    "replicas": get_replica_count("postgresql", environment),
    "selector": {
        "matchLabels": {
            "app": "postgresql"
        }
    },
    "template": {
        "metadata": {
            "labels": {
                "app": "postgresql"
            }
        },
        "spec": {
            "containers": [{
                "name": "postgresql",
                "image": "postgres:13",
                "env": [
                    {"name": "POSTGRES_DB", "value": "plebeian_academy"},
                    {"name": "POSTGRES_USER", "valueFrom": {"secretKeyRef": {"name": "db-secret"}, "key": "user"}},
                    {"name": "POSTGRES_PASSWORD", "valueFrom": {"secretKeyRef": {"name": "db-secret"}, "key": "password"}}
                ],
                "ports": [{"containerPort": 5432}],
                "volumeMounts": [{
                    "name": "postgres-storage",
                    "mountPath": "/var/lib/postgresql/data"
                }]
            }],
            "volumes": [{
                "name": "postgres-storage",
                "persistentVolumeClaim": {
                    "claimName": "postgres-pvc"
                }
            }]
        }
    }
}

// Backend API deployment
manifests["backend"] = {
    "apiVersion": "apps/v1",
    "kind": "Deployment",
    "metadata": {
        "name": "backend-deployment",
        "namespace": "plebeian-academy"
    },
    "spec": {
        "replicas": get_replica_count("backend", environment),
        "selector": {
            "matchLabels": {
                "app": "backend"
            }
        },
        "template": {

```



```

        "metadata": {
            "labels": {
                "app": "backend"
            }
        },
        "spec": {
            "containers": [{
                "name": "backend",
                "image": "plebeian-academy/backend:latest",
                "env": [
                    {"name": "ENVIRONMENT", "value": environment},
                    {"name": "DATABASE_URL", "valueFrom": {"secretKeyRef": {"name": "app-secret"}, "key": "database-url"}},
                    {"name": "REDIS_URL", "valueFrom": {"secretKeyRef": {"name": "app-secret"}, "key": "redis-url"}},
                    {"name": "SECRET_KEY", "valueFrom": {"secretKeyRef": {"name": "app-secret"}, "key": "secret-key"}}
                ],
                "ports": [{"containerPort": 5000}],
                "livenessProbe": {
                    "httpGet": {
                        "path": "/api/system/health",
                        "port": 5000
                    },
                    "initialDelaySeconds": 30,
                    "periodSeconds": 10
                },
                "readinessProbe": {
                    "httpGet": {
                        "path": "/api/system/health",
                        "port": 5000
                    },
                    "initialDelaySeconds": 5,
                    "periodSeconds": 5
                },
                "resources": {
                    "requests": {
                        "memory": "512Mi",
                        "cpu": "250m"
                    },
                    "limits": {
                        "memory": "1Gi",
                        "cpu": "500m"
                    }
                }
            }]
        }
    }

    // Frontend deployment
    manifests["frontend"] = {
        "apiVersion": "apps/v1",
        "kind": "Deployment",
        "metadata": {
            "name": "frontend-deployment",

```



```

        "kubernetes_sd_configs": [{"role": "pod"}],
        "relabel_configs": [
            {
                "source_labels": ["__meta_kubernetes_pod_annotation_prometheus_io_scrape"],
                "action": "keep",
                "regex": "true"
            }
        ]
    },
    {
        "job_name": "plebeian-academy-backend",
        "static_configs": [{"targets": ["backend-service:5000"]}],
        "metrics_path": "/api/system/metrics"
    },
    {
        "job_name": "plebeian-academy-blockchain",
        "static_configs": [{"targets": ["ganache-service:8545"]}]]
    ],
    "alerting": {
        "alertmanagers": [{"static_configs": [{"targets": ["alertmanager:9093"]}]]}
}

```

```

deploy_prometheus(prometheus_config)

```

```

// Deploy Grafana
grafana_dashboards = [
    "system-overview",
    "blockchain-metrics",
    "bot-performance",
    "network-health",
    "security-monitoring"
]

```

```

deploy_grafana(grafana_dashboards)

```

```

// Deploy Alertmanager
alertmanager_config = {
    "global": {
        "smtp_smarthost": "smtp.gmail.com:587",
        "smtp_from": "alerts@plebeian-academy.org"
    },
    "route": {
        "group_by": ["alertname"],
        "group_wait": "10s",
        "group_interval": "10s",
        "repeat_interval": "1h",
        "receiver": "web.hook"
    },
    "receivers": [
        {
            "name": "web.hook",
            "webhook_configs": [

```

```

        {
            "url": "http://alertmanager-webhook:5001/alerts",
            "send_resolved": TRUE
        }
    ]
}

]
}

deploy_alertmanager(alertmanager_config)

RETURN monitoring_setup_success()

FUNCTION configure_automated_backups():
    // Configure automated backup strategy

    backup_config = {
        "database": {
            "frequency": "daily",
            "retention": "30 days",
            "encryption": TRUE,
            "compression": TRUE,
            "storage": "s3://plebeian-academy-backups/database/"
        },
        "blockchain_data": {
            "frequency": "hourly",
            "retention": "7 days",
            "replication": TRUE,
            "storage": "s3://plebeian-academy-backups/blockchain/"
        },
        "application_data": {
            "frequency": "daily",
            "retention": "14 days",
            "storage": "s3://plebeian-academy-backups/application/"
        },
        "configuration": {
            "frequency": "weekly",
            "retention": "90 days",
            "storage": "s3://plebeian-academy-backups/config/"
        }
    }

    // Set up backup jobs
    FOR each backup_type, config IN backup_config:
        create_backup_cronjob(backup_type, config)

    // Set up backup monitoring
    setup_backup_monitoring()

    RETURN backup_configuration_success()

FUNCTION run_comprehensive_tests():
    // Run comprehensive test suite

```

```

test_results = {
    "unit_tests": run_unit_tests(),
    "integration_tests": run_integration_tests(),
    "security_tests": run_security_tests(),
    "performance_tests": run_performance_tests(),
    "end_to_end_tests": run_e2e_tests()
}

// Generate test report
test_report = generate_test_report(test_results)

// Check overall success
overall_success = all([
    test_results["unit_tests"].success,
    test_results["integration_tests"].success,
    test_results["security_tests"].success,
    test_results["performance_tests"].success,
    test_results["end_to_end_tests"].success
])

RETURN {
    "success": overall_success,
    "results": test_results,
    "report": test_report
}

FUNCTION monitor_system_health():
    // Continuous system health monitoring

    WHILE system_running:
        health_metrics = {
            "system_uptime": get_system_uptime(),
            "cpu_usage": get_cpu_usage(),
            "memory_usage": get_memory_usage(),
            "disk_usage": get_disk_usage(),
            "network_latency": measure_network_latency(),
            "database_health": check_database_health(),
            "blockchain_sync": check_blockchain_sync_status(),
            "bot_system_health": check_bot_system_health(),
            "security_status": check_security_status()
        }

        // Check for anomalies
        anomalies = detect_health_anomalies(health_metrics)

        // Send alerts if needed
        IF anomalies:
            send_health_alerts(anomalies)

        // Update monitoring dashboard
        update_monitoring_dashboard(health_metrics)

        // Log health status
        log_health_status(health_metrics)

```

```
wait(HEALTH_CHECK_INTERVAL)

// End of Comprehensive Pseudo Code
```

Summary

This comprehensive pseudo code covers the entire International Plebeian Academy holographic distributed system architecture, including:

1. **System Initialization** - Core system startup and configuration
2. **Authentication & Security** - Multi-modal biometric authentication with quantum security
3. **Blockchain File Verification** - Immutable file verification and upgrade token system
4. **Holographic Distribution** - Quantum-inspired data distribution across mesh network
5. **Biometric Administration** - Secure admin control with multiple biometric modalities
6. **Seven Division Bot System** - 35 specialized AI bots across 7 operational divisions
7. **Upgrade Propagation** - Worldwide automatic upgrade distribution and verification
8. **Frontend Application** - React.js PWA with real-time dashboard and biometric interface
9. **Backend API System** - Flask backend with comprehensive RESTful APIs
10. **Tribal Coin Blockchain** - Economic system with governance and mutual aid
11. **Infrastructure & Deployment** - Complete deployment, monitoring, and backup systems

The system is designed to be truly “everywhere and nowhere at once” with complete resilience, inviolable security, and automatic worldwide upgrade propagation as requested.